

实车挑战赛项目说明——编译基础说明

一、代码环境

操作系统	Ubuntu 20.04 LTS
中间件	ROS Noetic
语言	C++, Python($\geq 3.8, \leq 3.12$)
编译工具链	CMake($\geq 3.10.0$), Catkin
建议辅助工具	VSCode, Terminator

- 在本次实车挑战赛中，仿真及实车所采用的操作系统均为Ubuntu 20.04LTS，建议所有同学在个人电脑上统一安装此版本（尽量安装双系统，100G分区足矣；如实在无法安装双系统，也可以考虑使用虚拟机）。镜像可以前往官网或清华镜像站([清华大学开源软件镜像站 | Tsinghua Open Source Mirror](#))下载。
- 所采用的通信中间件为ROS Noetic，可参照以下链接等进行安装
 - [【安装】Ubuntu20.04下安装ROS的完整过程（内含已装好ROS的虚拟机、虚拟机创建过程、ROS安装过程及全过程录屏） ros虚拟机现成的-CSDN博客](#)
 - [ROS 安装详细教程—— Ubuntu20.04 LTS 安装 - pqppl - 博客园](#)
- 所要求的编程语言包含C++与Python，Python建议版本不低于3.8，不高于3.12。如各位同学有Anaconda等虚拟环境的使用需求，则有如下建议：
 - 如果是在全新的系统中安装，建议先安装ROS，确保ROS下python节点可以启动后，再安装Anaconda
 - 在基于ROS完成代码开发与调试时，建议在.bashrc文件中注释掉Anaconda默认启动语句；在需要使用conda环境时再临时打开，或可参考[Ubuntu 修改~/.bashrc终端选择是否使用annconda环境 bashrc conda-CSDN博客](#)
- 如果所安装的Ubuntu 20.04操作系统未自带CMake，可以考虑安装，并采用如下语句验证版本是否大于3.10.0

```
cmake --version
```

- 基于助教们的过往开发经验，这里向大家建议采用如下工具
 - VSCode，宇宙最强编辑器。虽然在Ubuntu下有Clion、Sublime等多种或集成或轻量的编辑器选择，但是考虑到项目复杂程度，多样且自由的插件可以为同学们提供最大程度的便利。建议去VSCode官方网站上下载，不要在Ubuntu应用商店中下载。这里给出一些建议的插件清单，同学们可以视情况添加。
 - Terminator，终端终结者。Ubuntu一般自带的终端软件为Gnome Terminal，已经足够好用。但是考虑到我们的工程较为复杂，完整的实现各部分功能需要多个终端同时启动各个节点并调试。这里向大家强烈推荐Terminator终端终结者，可以在一个界面中通过上下左右分屏等方式同时展示多个终端，熟练掌握快捷键之后可以大幅提升启动代码的速度，非常方便。[Ubuntu下terminator的安装/配置/默认启动/使用 terminator 默认-CSDN博客](#)

插件名称	功能	相关链接
ROS	强烈推荐，调试利器！ 提供了许多对ROS环境的支持，包括快捷创建包、启动roscore等，但最主要的推荐原因是提供了ROS环境的下的debugger，让你可以 断点调试 程序。	【ROS】在VScode中 ROS Debug 配置方法非常详细版
Todo Tree	代码太复杂，需要留些标记提醒自己？尝试多用TODO和FIXME！这个插件可以快速向你展示当前工程中有哪些标签需要你关注	Vscode 插件：Todo Tree 简介和使用方法-CSDN博客
CMake与CMake Tools	这个插件其实是你想在VSCode下调试C++程序必须的插件。在这个项目中了解并尝试修改CMakeList可能是你必经的环节，这个插件也可以给你的编写提供一些提示。	VSCode + CMake：打造高效C/C++开发环境的指南 vscode cmake-CSDN博客

二、项目结构（含ROS基础）

Note

此部分关于ROS基础的相关描述仅代表个人观点，且为便于初学者理解进行了一定简化。严谨使用与表述请参照ROS官方文档。

2.1 Workspace与编译

每位同学都应有一份名为avdp_path_planning_base的文件，解压后应该包含一个名为src的文件夹。了解过ROS的同学应当清楚，avdp_path_planning_base即为工作空间workspace，而src则是代码空间。我们首先需要对工作空间进行初始化

```
cd avdp_path_planning_base/src
catkin_init_workspace
```

这时在src文件夹中会出现一个CMakeLists.txt的软链接。这说明我们已成功使avdp_path_planning_base成为一个工作空间，且可以在下一步对src中的代码进行编译。

在src代码空间中，每一个文件夹均是一个ROS的功能包。我们提供了4个功能包，分别实现了环境仿真(cssr)、轨迹跟踪控制(cav_control)、**轨迹规划(cav_traj_gen)** 和发布参考路径以替代上层规划(cav_path_pub)的功能。此外，还有一个额外的功能包cav_msgs，定义了在此工作空间中各节点交互时所采用的消息类型。

接下来我们可以尝试编译工程。

```
cd avdp_path_planning_base
catkin_make
```

如果终端显示顺利编译至100%，我们可以看到在工作空间中额外出现了两个文件夹，分别为devel(开发空间)和build(编译空间)。devel空间放置开发过程中的中间文件，而build空间存放有可执行文件。我们每一次对C++程序修改后，都需要重新编译才能执行修改后的结果（这是常识）。

如果发现编译过程无法顺利进行，出现报错，可以排查以下可能的情况：

现象1：显示某一个包在编译的时候缺少某些msg的定义

原因：在第一次编译工程时，我们没有明确指定各个包间的编译顺序和依赖，它们的编译顺序在不同的电脑中可能不一致。而我们的四个功能包明确依赖于消息类型包cav_msgs，所以应先编译cav_msgs，再编译其他四个功能包，才可以保证不会缺少msg定义。

解决方案：

1. **方案1（投机取巧法）：**直接再执行一次或多次catkin_make
2. **方案2（依次编译法）：**将除了cav_msgs以外的功能包移动至上一级目录，使他们与src同级后，进行编译，此时将只对cav_msgs进行编译。编译通过后，再将其他四个功能包移动回src内，再次进行编译。

现象2：Unable to find either executable 'empy' or Python module 'em'... try installing the package 'python3-empy'

原因：这个报错往往出现于系统中已安装了conda的情况，此时编译时系统默认寻找的是conda的base环境中的python，而不是系统中安装的默认python。

解决方案：在编译时添加后缀，指定使用哪个python编译

```
catkin_make -DPYTHON_EXECUTABLE=/usr/bin/python3
```

当然你需要确认一下，你的电脑上是否有这个python路径。

🔥 Important

最后再次提醒，务必退出conda环境再编译或执行程序。

2.2 Package与开发

现在聚焦到具体的一个功能包中。对于每一个功能包，可能会包含如下内容：

1. `config`：功能包的配置文件，用户创建。
2. `include`：需要用到的头文件。
3. `scripts`：可以直接运行的Python脚本。
4. `src`：需要编译的C++代码。
5. `launch`：所有启动文件。
6. `msg`：自定义的消息类型。
7. `srv`：自定义的服务类型。
8. `action`：自定义的动作指令。

一定会包含以下两个文件：

1. `CMakeLists.txt`：Catkin编译器编译功能包的规则。
2. `package.xml`：功能包清单，可得到该package的名称、版本号、信息描述、作者信息和许可信息等。更重要的，`<build_depend> ... </build_depend>` 标签定义了代码编译所依赖的其它功能包，`<exec_depend> ... </exec_depend>` 标签定义了可执行程序运行时所依赖的其它功能包。

对于我们当前的工程，每一个功能包都包含有预定义的launch文件，用以启动各个节点。对于本次实践中不要求开发的cssr、cav_control、cav_path_pub，同学们**可以仅浏览launch目录下的文件，并选择性修改其参数**，以实现不同的效果。

对于本次实践的核心功能包cav_traj_plan，需要同学们详细浏览并完成其src下的代码。

2.3 节点运行

在每一个package中，可以包含多个节点，每个节点可以实现某个特定的功能并与其他ros节点进行交互。通过多个节点可以共同组成一个庞大的网络，实现复杂的功能。在所有节点中，ROS1最为特殊的节点是核心节点roscore（ROS2的去中心化架构已无需roscore）。

roscore是ros的核心服务，可以注册各个节点、存储所有的ros参数并记录日志。此外它还是一个信使，负责连接各个节点的通信。在运行其他节点前必须确保roscore的运行，而且只存在一个roscore。

roscore一般启动方式有两种：

1. 在终端中输入 `roscore`
2. 在未手动启动roscore的情形下，通过roslaunch启动了某一个节点，会自动启动一个roscore。这个roscore有可能会随着此终端下的节点关闭而关闭。因此在多节点运行时，如果不小心关闭这个启动了roscore的roslaunch节点，可能会导致后续的其他节点无法正常通信，甚至可能会出现不同的节点非法连接到了不同的roscore上，导致通信不通的情况。因此，**不推荐!!!**。建议大家开的第一个终端就是启动roscore，并且不要关闭这个终端。

对于一般功能包中的节点，有roslaunch和roslaunch两种启动方式。其中roslaunch仅能启动单个节点，而且无法在启动时设置参数，只能通过额外输入roslaunch调整参数。而roslaunch可以启动多package内的多node，同时支持参数的设置，非常方便。因此，我们在每个功能包内都写有launch文件，方便大家运行。

```
roslaunch [package] [filename.launch]
```

三、上手实践

通过上面的介绍，相信大家已经大致了解如何启动整个项目。接下来我们将对整个过程进行整理。

3.0 准备工作

在网络学堂上下载cav_deploy.zip文件

修改 `/opt` 目录的权限为可读写

```
cd /  
sudo chmod 777 /opt
```

将cav_deploy.zip拷贝至/opt 目录下，并解压，得到/opt/cav_deploy

3.1 初始化工作空间并编译

```
cd avdp_path_planning_base/src  
catkin_init_workspace  
cd ..  
catkin_make
```

3.2 启动roscore

```
roscore
```

3.3 运行节点

在每一个终端中，为了使ros能够检索到我们创建的功能包，我们需要在工作空间内执行如下命令

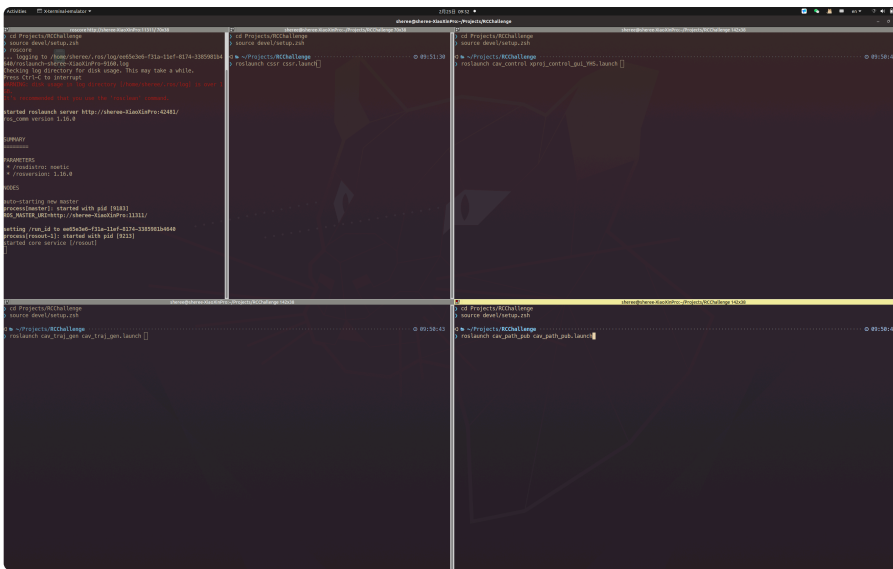
```
source devel/setup.bash
```

这是非常关键的一步，每到一个新的terminal都需要source！你也可以选择在.bashrc文件中写入这个命令，每次开启新的terminal后会自动执行

这样当前ros系统的package列表就包含了我们所创建的功能包，我们可以通过如下指令查看确认

```
rospack list
```

接下来，除了已有的roscore终端外，创建4个新的终端，分别用来启动各个节点。



ed33b00a53d534521cf879c5fbe9d22.png

3.3.1 仿真

```
cd avdp_path_planning_base
source devel/setup.bash
roslaunch cssr cssr.launch
```

3.3.2 跟踪控制

```
cd avdp_path_planning_base
source devel/setup.bash
roslaunch cav_control cav_control_YHS.launch
```

3.3.3 轨迹规划

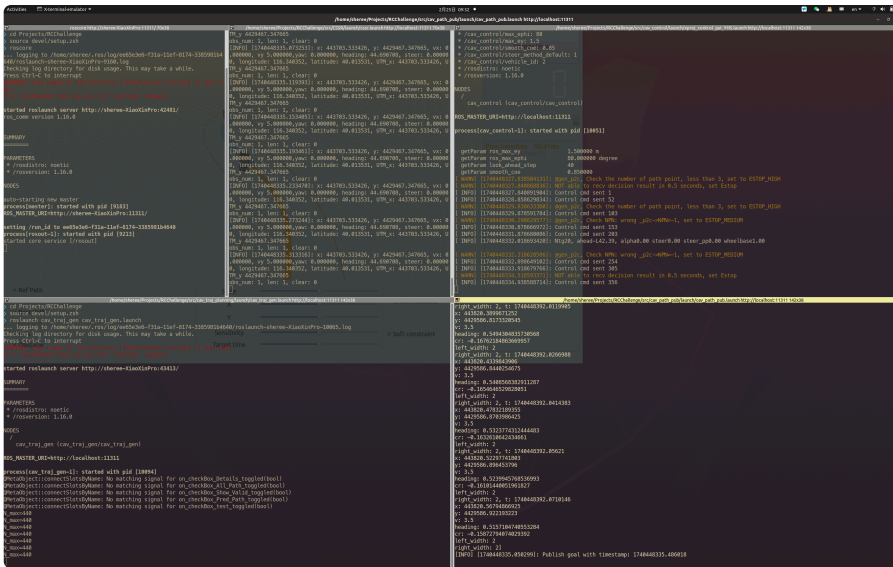
```
cd avdp_path_planning_base
source devel/setup.bash
roslaunch cav_traj_gen cav_traj_gen.launch
```

3.3.4 参考路径发布

```
cd avdp_path_planning_base
source devel/setup.bash
roslaunch cav_path_pub cav_path_pub.launch
```

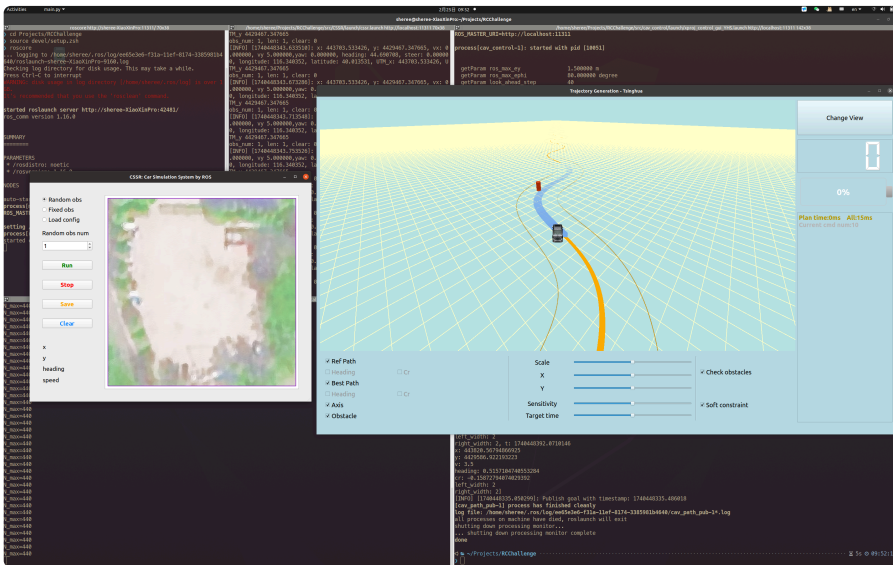
需要**注意**的是，前三个程序设置了循环运行，除非Ctrl+C终止程序或关闭图形化界面，否则不会停止；最后一个参考路径发布程序则仅执行一段时间后自动退出，即发布参考路径后就退出。因此需要在轨迹规划程序启动后，再启动参考路径发布程序，才能保证轨迹规划程序接收到发布的参考路径，否则无法收到。

运行后的各终端如下↓



99a902500101d9aff050e8e19d236ae.png

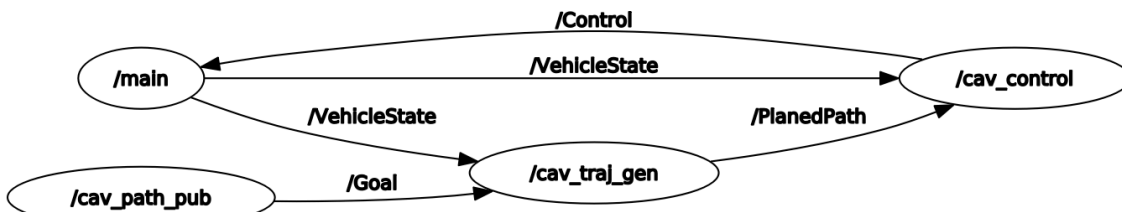
应该包含两个图形化界面↓



83ca874640c81f9c488c1ee9b776439.png

四、整体架构

在这个图中，椭圆形的是节点，箭头上的是各节点间通讯使用的topic名称。在这个图中我们可以看到明确的信息流，以便更好地理解整体的架构。



Pasted image 20250228191539.png